

PREMIUM STUDY HANDBOOK

SQL Interview Handbook

MySQL · Relational Databases · Data Analytics

A complete, interview-focused guide covering every SQL concept with syntax, examples, outputs, common mistakes, and 100+ interview Q&A — built around one connected sample e-commerce database (**ecom**).

Beginner Friendly | Interview Ready | Quick Revision

Edition 2026

Table of Contents

Chapter 1: Introduction to Databases & SQL	3
1.1 Why Not Excel or Text Files?	
1.2 What Is a Database?	
1.3 Where SQL Fits In	
1.4 Relational vs Non-Relational Databases	
Chapter 2: Getting Started - Our Sample Database	5
2.1 Creating & Using a Database	
2.2 Resetting the Database	
2.3 The Sample Table: orders	
2.4 Inserting Sample Rows	
Chapter 3: Data Types & Table Design	7
3.1 Commonly Used MySQL Data Types	
3.2 Creating a Table	
3.3 Altering, Renaming & Dropping	
Chapter 4: CRUD - INSERT, SELECT, UPDATE, DELETE	9
4.1 INSERT, 4.2 SELECT, 4.3 WHERE, 4.4 AND/OR/Sort, 4.5 UPDATE, 4.6 DELETE	
Chapter 5: ALTER, DROP & Transactions	13
5.1 ALTER TABLE 5.2 DROP vs DELETE vs TRUNCATE 5.3 Transactions	
Chapter 6: Constraints, Primary Key & AUTO_INCREMENT	16
6.1 Constraints Overview 6.2 Example Table 6.3 PRIMARY KEY	
Chapter 7: Sorting, Limiting, Functions & Logical Operators	19
7.1 ORDER BY & LIMIT 7.2 Aggregate Functions 7.3 Scalar 7.4 Date 7.5 Logical Operators	
Chapter 8: Foreign Keys, ON DELETE & JOINS	23
8.1 Foreign Keys 8.2 ON DELETE Options 8.3 JOINS	
Chapter 9: Indexes & Views	28
9.1 Indexes 9.2 Views	
Chapter 10: GROUP BY, HAVING & ROLLUP	31
10.1 GROUP BY 10.2 HAVING 10.3 ROLLUP	
Chapter 11: Subqueries & UNION	35
11.1 Subqueries 11.2 UNION & UNION ALL	
Chapter 12: Stored Procedures & Triggers	38
12.1 Stored Procedures 12.2 Triggers	
Chapter 13: 100+ SQL Interview Questions & Answers	42
Section A: Conceptual (Q1-30) Section B: Coding (Q31-50) Section C: Advanced (Q51-100)	
Chapter 14: Cheat Sheets & Quick Reference	65
SQL Execution Order Aggregate Cheat Sheet JOIN Cheat Sheet Window Functions	

Chapter 1: Introduction to Databases & SQL

Foundations every interviewer expects you to know in one line.

1.1 Why Not Excel or Text Files?

- Excel becomes very slow once data grows into lakhs/millions of rows.
- No multi-user access control → risk of accidental overwrite.
- Text files have no structure, no data types, no relationships.
- Neither supports complex queries, automation, or guaranteed consistency.

1.2 What Is a Database?

Simple Definition

An organized collection of data stored electronically so it can be accessed, managed and updated quickly and safely.

- Fast even with millions of rows.
- Enforces structure, rules and consistency.
- Supports multiple users at the same time, with permissions.

1.3 Where SQL Fits In

SQL = Structured Query Language. It is the language used to **store, retrieve, update and analyze** data inside a database. We will use **MySQL** throughout this handbook.

Memory Trick

SQL = "Speak to your Quantities of data, Logically." Or just remember

C-R-U-D

: Create, Read, Update, Delete — everything else is built on these four actions.

1.4 Relational vs Non-Relational Databases

Feature	Relational (SQL)	Non-Relational (NoSQL)
Data format	Tables (rows & columns)	JSON, key-value, graph, column
Schema	Fixed	Flexible / dynamic
Relationships	Supported via foreign keys	Limited / manual
Consistency	Strong (ACID)	Often eventual
Query language	Standard SQL	Database-specific
Best for	Analytics, BI, structured apps	Real-time, high-scale, unstructured data
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Redis, Cassandra, Neo4j

Common Interview Questions

Q: Why choose a relational database over NoSQL for analytics?

A: Relational DBs enforce a fixed schema and strong consistency, and SQL is the industry-standard query language for aggregation and reporting.

Q: What does ACID mean?

A: Atomicity, Consistency, Isolation, Durability — guarantees that transactions are processed reliably.

Quick Revision - Chapter 1

- Database = organized, fast, secure data store.
- SQL = language to talk to relational databases.

- Relational = tables + fixed schema; Non-relational = flexible schema.
- MySQL is our practice tool throughout this handbook.

Beginner Mistake

Thinking SQL and MySQL are the same thing. SQL is the *language*; MySQL is one *database product* that understands SQL.

Chapter 2: Getting Started - Our Sample Database

Every example in this handbook uses this one database: `ecom`.

2.1 Creating & Using a Database

```
CREATE DATABASE ecom;    -- creates a new empty database named ecom
USE ecom;               -- tells MySQL: run all next queries inside ecom
```

Keyword	Meaning
<code>CREATE DATABASE</code>	Makes a new, empty database container.
<code>USE</code>	Switches the "active" database for the session.

Common Mistake

Forgetting `USE ecom;` — MySQL won't know where to create tables and will throw "No database selected."

2.2 Resetting the Database (Start Fresh)

```
DROP DATABASE IF EXISTS ecom; -- deletes ecom only if it already exists, no error otherwise
CREATE DATABASE ecom;
USE ecom;
```

2.3 The Sample Table: `orders`

```
CREATE TABLE orders (
  order_id      INT PRIMARY KEY AUTO_INCREMENT, -- unique row id, auto numbered
  customer_name VARCHAR(100),                  -- buyer's name
  city          VARCHAR(50),                    -- delivery city
  product       VARCHAR(100),                  -- product name
  category      VARCHAR(50),                  -- product category
  quantity      INT,                          -- units ordered
  price_per_unit DECIMAL(10,2),                -- price of 1 unit
  discount_percent INT,                       -- discount applied
  order_date    DATE,                         -- when order was placed
  delivery_date DATE,                         -- when order was delivered (NULL if not yet)
  payment_mode  VARCHAR(30),                  -- Cash / UPI / Card
  order_status  VARCHAR(30),                  -- Delivered / Pending / Cancelled
  rating        INT,                          -- customer rating 1-5 (NULL if not rated)
);
```

2.4 Inserting Sample Rows

```
INSERT INTO orders
(customer_name, city, product, category, quantity, price_per_unit,
 discount_percent, order_date, delivery_date, payment_mode, order_status, rating)
VALUES
('Amit Sharma', 'Delhi', 'Laptop', 'Electronics', 1, 65000, 10, '2025-01-05', '2025-01-08', 'Credit Card',
'Delivered', 5),
('Neha Verma', 'Mumbai', 'Headphones', 'Electronics', 2, 2500, 0, '2025-01-10', '2025-01-12', 'UPI',
'Delivered', 4),
('Rahul Khan', 'Delhi', 'Office Chair', 'Furniture', 1, 12000, 15, '2025-01-12', '2025-01-20', 'Debit Card',
'Delivered', 5),
('Arjun Mehta', 'Ahmedabad', 'Smartphone', 'Electronics', 1, 30000, 5, '2025-01-18', NULL, 'UPI',
'Cancelled', NULL);
-- ...more rows; full script available in Chapter 4
```

Why This Table?

The `orders` table has numbers (price, quantity), text (city, product), dates (order_date) and NULLs (delivery_date) — perfect for practicing every SQL concept in one place.

Quick Revision - Chapter 2

- `CREATE DATABASE` / `USE` / `DROP DATABASE IF EXISTS` are your starting commands.
- One sample table `orders` is reused in every chapter ahead.

Chapter 3: Data Types & Table Design

3.1 Commonly Used MySQL Data Types

Type	Stores	Example Column
INT	Whole numbers	age, quantity, ids
VARCHAR (n)	Text up to n characters	name, email, city
DECIMAL (p, s)	Exact decimal numbers (p=total digits, s=after decimal)	price, salary
BOOLEAN	True / False	is_active
DATE	Date only (YYYY-MM-DD)	signup_date
DATETIME	Date + time together	created_at
ENUM ('a', 'b', ...)	One value from a fixed list	status

Memory Trick

I-V-D-B-D-D-E

:

I

nteger

V

alues

D

on't

B

ore

D

ata

D

esign

E

ngineers." (INT, VARCHAR, DECIMAL, BOOLEAN, DATE, DATETIME, ENUM)

3.2 Creating a Table

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(150),  
  age INT,  
  phone VARCHAR(15),  
  is_active BOOLEAN,  
  signup_date DATE,  
  created_at DATETIME,  
  total_spent DECIMAL(10,2)  
);
```

3.3 Altering, Renaming & Dropping

```
DROP TABLE customers;           -- deletes table + data permanently  
ALTER TABLE customers RENAME TO clients;  -- renames the table
```

Command	What Happens
ALTER TABLE	Changes structure — data stays

<code>DELETE FROM table</code>	Deletes data — structure stays
<code>DROP TABLE</code>	Deletes data <i>and</i> structure

Common Interview Questions

Q: Difference between CHAR and VARCHAR?

A: CHAR is fixed-length (always pads with spaces), VARCHAR is variable-length (stores only what's needed) – VARCHAR saves space for variable text.

Q: Difference between DROP, DELETE and TRUNCATE?

A: DROP removes the table entirely; DELETE removes rows (can use WHERE, can rollback); TRUNCATE removes all rows fast and resets AUTO_INCREMENT, but cannot use WHERE.

Beginner Mistake

Using `FLOAT` for money. Always use `DECIMAL` for prices/salary — FLOAT can cause rounding errors.

Quick Revision - Chapter 3

- Pick the right data type: INT for numbers, VARCHAR for text, DECIMAL for money, DATE/DATETIME for time.
- ALTER changes structure, DROP destroys everything.

Chapter 4: CRUD - INSERT, SELECT, UPDATE, DELETE

CRUD = Create, Read, Update, Delete
— the four basic things you can do to data.

Operation	SQL Command
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

4.1 INSERT - Adding Data

Syntax

```
INSERT INTO table_name (col1, col2, ...)  
VALUES (val1, val2, ...);
```

Example: Single Row

```
INSERT INTO customers (name, email, age, phone, is_active, signup_date, created_at, total_spent)  
VALUES ('Amit Sharma', 'amit@gmail.com', 28, '9876543210', TRUE, '2025-01-10', '2025-01-10 10:30:00',  
12000.50);
```

Example: Multiple Rows

```
INSERT INTO customers (name, email, age, phone, is_active, signup_date, created_at, total_spent)  
VALUES  
( 'Neha Verma', 'neha@gmail.com', 25, '9123456789', TRUE, '2025-01-12', '2025-01-12 09:15:00', 5400.00),  
( 'Rahul Khan', 'rahul@gmail.com', 32, '9988776655', FALSE, '2025-01-15', '2025-01-15 14:20:00', 0.00);
```

Common Mistake

Column order in `INSERT INTO table(...)` must match the order of values exactly. Mixing the order without naming columns inserts wrong data into wrong columns.

4.2 SELECT - Reading Data

```
SELECT * FROM orders;           -- all columns, all rows  
SELECT customer_name, product, city FROM orders;  -- only chosen columns
```

Output Explanation: `SELECT *` returns every column in the table's defined order; naming columns returns only those, in the order you listed them.

4.3 WHERE - Filtering Rows

```
SELECT * FROM orders WHERE city = 'Delhi';  
SELECT customer_name, product, price_per_unit FROM orders WHERE price_per_unit > 5000;  
SELECT customer_name, product, price_per_unit FROM orders WHERE price_per_unit != 5000;
```

NULL Trap - Important Interview Topic

`NULL` means "unknown", not zero or empty. So `column = NULL` NEVER returns rows — always use `IS NULL` / `IS NOT NULL`.

```
SELECT * FROM orders WHERE delivery_date IS NULL;  -- correct way to find missing dates
```

4.4 AND / OR / Sorting

```
SELECT * FROM orders WHERE city = 'Delhi' AND order_status = 'Delivered';
SELECT customer_name, order_date, price_per_unit
FROM orders ORDER BY order_date DESC; -- latest orders first
```

4.5 UPDATE - Modifying Data

```
UPDATE orders SET order_status = 'Delivered' WHERE order_id = 10;

UPDATE orders
SET discount_percent = 10, rating = 4
WHERE customer_name = 'Neha Verma'; -- update multiple columns at once

UPDATE orders SET order_status = 'Cancelled' WHERE order_status = 'Pending';
```

Golden Rule

Always run a `SELECT` with the same `WHERE` first to preview which rows will change, before running `UPDATE` or `DELETE`.

4.6 DELETE - Removing Data

```
DELETE FROM orders WHERE order_id = 5;
DELETE FROM orders WHERE order_status = 'Cancelled';
DELETE FROM orders WHERE order_date < '2025-01-10';
DELETE FROM orders; -- removes ALL rows but keeps the table structure -- use carefully!
```

Common Interview Questions

Q: What happens if you forget WHERE in an UPDATE/DELETE?

A: Every single row in the table gets updated or deleted – one of the most common production disasters.

Q: Why does WHERE rating = NULL return nothing?

A: Because NULL is "unknown" – any comparison with NULL returns UNKNOWN, not TRUE. Use IS NULL.

Quick Revision - Chapter 4

- INSERT adds rows, SELECT reads, UPDATE modifies, DELETE removes.
- Always pair UPDATE/DELETE with WHERE (preview using SELECT first).
- NULL needs IS NULL / IS NOT NULL, never `=` or `!=`.

Coding Practice - Chapter 4

1. Select all orders from 'Mumbai' with rating above 3.
2. Update all 'Pending' orders in 'Delhi' to 'Cancelled'.
3. Delete all orders where delivery_date IS NULL and order_status = 'Cancelled'.

Chapter 5: ALTER, DROP & Transactions

5.1 ALTER TABLE

```
ALTER TABLE orders ADD COLUMN delivery_partner VARCHAR(50); -- new column, NULL for existing rows
ALTER TABLE orders MODIFY price_per_unit DECIMAL(12,2); -- change data type/size
ALTER TABLE orders RENAME COLUMN city TO customer_city; -- rename, data kept
ALTER TABLE orders DROP COLUMN rating; -- removes only that column
```

5.2 DROP TABLE vs DELETE vs TRUNCATE

Command	Data	Structure	Rollback?	WHERE allowed?
DELETE FROM orders;	Removed	Kept	Yes (in transaction)	Yes
TRUNCATE TABLE orders;	Removed (fast)	Kept	No	No
DROP TABLE orders;	Removed	Removed	No	No

```
DROP TABLE IF EXISTS orders; -- avoids an error if the table doesn't exist
```

5.3 Transactions

- A transaction = a group of statements that succeed together or fail together.
- By default MySQL auto-saves (commits) every statement instantly.
- Turn this off to control exactly when changes become permanent.

```
SET autocommit = 0; -- stop auto-saving every statement

UPDATE orders SET order_status = 'Cancelled' WHERE order_id = 3; -- temporary change

SELECT order_id, order_status FROM orders WHERE order_id = 3; -- preview it

COMMIT; -- saves permanently, cannot be undone
-- OR
ROLLBACK; -- undoes everything since the last COMMIT
```

Transaction Flow (memorize this order)

SET autocommit = 0 → INSERT/UPDATE/DELETE → COMMIT (save) or ROLLBACK (undo)

Note

Transactions only work with transactional engines like **InnoDB** (MySQL's default). SELECT statements never need COMMIT/ROLLBACK.

Common Interview Questions

Q: What is the difference between COMMIT and ROLLBACK?

A: COMMIT permanently saves all changes in the current transaction; ROLLBACK undoes all changes since the last COMMIT.

Q: Can you ROLLBACK after TRUNCATE?

A: No – TRUNCATE is a DDL operation and auto-commits; it cannot be rolled back.

Quick Revision - Chapter 5

- ALTER changes structure without losing data.
- DROP > TRUNCATE > DELETE in terms of destructiveness/speed.
- Transactions = SET autocommit=0 → changes → COMMIT/ROLLBACK.

Chapter 6: Constraints, Primary Key & AUTO_INCREMENT

6.1 Constraints Overview

Constraint	Purpose
UNIQUE	No duplicate values allowed in that column.
NOT NULL	Column can never be empty/NULL.
CHECK	Value must satisfy a condition (e.g. age >= 18).
DEFAULT	Auto-fills a value when none is given.
PRIMARY KEY	Uniquely identifies each row; implies UNIQUE + NOT NULL.
FOREIGN KEY	Links to a primary key in another table (Chapter 8).

6.2 Example Table with Constraints

```
CREATE TABLE employees (  
  emp_id      INT PRIMARY KEY AUTO_INCREMENT,  
  email       VARCHAR(150) UNIQUE,           -- no two employees share an email  
  name        VARCHAR(100) NOT NULL,         -- name is compulsory  
  age         INT CHECK (age >= 18),          -- must be an adult  
  department  VARCHAR(50) DEFAULT 'General', -- default department  
  salary      DECIMAL(10,2) CHECK (salary > 0),  
  joining_date DATE DEFAULT (CURRENT_DATE)    -- defaults to today  
);
```

Testing Each Constraint

```
-- UNIQUE: this will FAIL on a 2nd identical email  
INSERT INTO employees (email, name, age, salary) VALUES ('amit@company.com', 'Amit Sharma', 25, 45000);  
  
-- NOT NULL: this will FAIL because name is missing  
INSERT INTO employees (email, age, salary) VALUES ('neha@company.com', 24, 40000);  
  
-- CHECK: this will FAIL because age < 18  
INSERT INTO employees (email, name, age, salary) VALUES ('rahul@company.com', 'Rahul Khan', 16, 30000);  
  
-- DEFAULT: department becomes 'General', joining_date becomes today  
INSERT INTO employees (email, name, age, salary) VALUES ('pooja@company.com', 'Pooja Nair', 26, 50000);
```

6.3 PRIMARY KEY & AUTO_INCREMENT

- PRIMARY KEY uniquely identifies each row; cannot be NULL or duplicate.
- AUTO_INCREMENT auto-generates the next number (1, 2, 3...).
- A table can have only **one** PRIMARY KEY.

```
INSERT INTO employees (email, name) VALUES ('amit@company.com', 'Amit Sharma');  
-- MySQL assigns emp_id = 1 automatically  
  
-- This FAILS: a table can only have one PRIMARY KEY  
ALTER TABLE employees ADD PRIMARY KEY (email);
```

Common Interview Questions

Q: Difference between PRIMARY KEY and UNIQUE KEY?

A: A table can have only ONE primary key (no NULLs allowed), but MANY unique keys (each allows one NULL value).

Q: What is a composite primary key?

A: A primary key made of two or more columns together, used when no single column is unique on its own.

Beginner Mistake

Forgetting that `CHECK` constraints are silently ignored in very old MySQL versions (<8.0.16) — always confirm the MySQL version in interviews if asked.

Quick Revision - Chapter 6

- UNIQUE, NOT NULL, CHECK, DEFAULT = data quality rules.
- PRIMARY KEY = UNIQUE + NOT NULL, one per table, often AUTO_INCREMENT.

Chapter 7: Sorting, Limiting, Functions & Logical Operators

7.1 ORDER BY & LIMIT

```
SELECT * FROM orders ORDER BY price_per_unit;           -- ascending (default)
SELECT * FROM orders ORDER BY price_per_unit DESC;      -- descending
SELECT * FROM orders ORDER BY city ASC, price_per_unit DESC; -- sort by city, then price within city
SELECT * FROM orders LIMIT 5;                          -- first 5 rows only
SELECT customer_name, product, price_per_unit
FROM orders ORDER BY price_per_unit DESC LIMIT 3;       -- top 3 most expensive orders
SELECT * FROM orders ORDER BY order_date LIMIT 5 OFFSET 5; -- skip first 5, return next 5 (pagination)
```

Interview Shortcut

"Top-N" or "Nth highest" questions almost always use `ORDER BY ... DESC LIMIT N` (with OFFSET for pagination).

7.2 Aggregate Functions (work on many rows → 1 result)

```
SELECT COUNT(*) FROM orders; -- total number of orders
SELECT SUM(quantity * price_per_unit) AS total_revenue FROM orders;
SELECT AVG(price_per_unit) FROM orders; -- average price
SELECT MIN(price_per_unit), MAX(price_per_unit) FROM orders; -- cheapest & costliest
```

7.3 Scalar Functions (work row by row)

```
SELECT customer_name, ROUND(price_per_unit, 0) FROM orders; -- round to nearest whole number
SELECT UPPER(customer_name), LOWER(city) FROM orders; -- change text case
SELECT customer_name, LENGTH(customer_name) FROM orders; -- character count
```

7.4 Date Functions

```
SELECT CURRENT_DATE; -- today's date
SELECT order_id, DATEDIFF(delivery_date, order_date) AS delivery_days FROM orders;
SELECT * FROM orders WHERE YEAR(order_date) = 2025; -- filter by year
```

7.5 Logical Operators

```
SELECT * FROM orders WHERE city IN ('Delhi', 'Mumbai', 'Bangalore'); -- replaces multiple ORs
SELECT * FROM orders WHERE payment_mode NOT IN ('Cash', 'UPI');
SELECT * FROM orders WHERE price_per_unit BETWEEN 1000 AND 10000; -- inclusive range
SELECT * FROM orders WHERE customer_name LIKE 'A%'; -- starts with A
SELECT * FROM orders WHERE product LIKE '%Table%'; -- contains "Table"
SELECT * FROM orders WHERE city LIKE 'D_lhi'; -- exactly 1 wildcard char (matches Delhi)
```

Symbol	Meaning
<code>%</code>	Any number of characters (including zero)
<code>_</code>	Exactly one character

Common Interview Questions

Q: Difference between WHERE and HAVING?

A: WHERE filters rows before grouping; HAVING filters groups after aggregation (covered in Chapter 10).

Q: Is BETWEEN inclusive or exclusive?

A: Inclusive on both ends – BETWEEN 1000 AND 10000 includes 1000 and 10000.

Beginner Mistake

Writing `LIKE 'Table'` instead of `LIKE '%Table%'` expecting a "contains" match – without `%`, LIKE behaves like an exact match.

Quick Revision - Chapter 7

- ORDER BY sorts, LIMIT restricts row count.
- Aggregate = COUNT/SUM/AVG/MIN/MAX (many rows → one value).
- IN, BETWEEN, LIKE replace long OR chains and enable pattern matching.

Coding Practice - Chapter 7

1. Find the 2nd most expensive order using ORDER BY + LIMIT + OFFSET.
2. Find total revenue per order using SUM(quantity * price_per_unit).
3. Find all customers whose name ends with "a" using LIKE.

Chapter 8: Foreign Keys, ON DELETE & JOINS

8.1 Foreign Keys

- A FOREIGN KEY links one table to another and blocks invalid references.
- Prevents an order from pointing to a seller that doesn't exist.

```
CREATE TABLE sellers (  
  seller_id INT PRIMARY KEY AUTO_INCREMENT,  
  seller_name VARCHAR(100) UNIQUE NOT NULL,  
  city VARCHAR(50)  
);  
  
ALTER TABLE orders ADD COLUMN seller_id INT;  
  
ALTER TABLE orders  
ADD CONSTRAINT fk_orders_seller  
FOREIGN KEY (seller_id) REFERENCES sellers(seller_id); -- seller_id must exist in sellers
```

```
-- Works: seller 1 exists  
INSERT INTO sellers (seller_name, city) VALUES ('TechWorld', 'Delhi');  
INSERT INTO orders (seller_id, product, quantity, price_per_unit) VALUES (1, 'Laptop', 1, 65000);  
  
-- FAILS: seller 999 doesn't exist  
INSERT INTO orders (seller_id, product, quantity, price_per_unit) VALUES (999, 'Laptop', 1, 65000);
```

8.2 ON DELETE Options

Option	Effect when parent (seller) row is deleted	Use Case
CASCADE	Deletes all related child (order) rows too	Marketplace shutdown
SET NULL	Keeps orders, sets seller_id to NULL	Seller leaves, history kept
RESTRICT (default)	Blocks deletion if any order exists	Prevent accidental loss

```
ALTER TABLE orders DROP FOREIGN KEY fk_orders_seller;  
ALTER TABLE orders  
ADD CONSTRAINT fk_orders_seller  
FOREIGN KEY (seller_id) REFERENCES sellers(seller_id)  
ON DELETE CASCADE; -- deleting a seller deletes their orders too
```

Memory Trick

"
C
ASCAD
C
hain-deletes.
S
ET NULL
S
aves the child.
R
ESTRICT
R
efuses."

8.3 JOINS

JOINS combine rows from two or more tables using a related column (here, `seller_id`).

JOIN Type	Returns
INNER JOIN	Only matching rows in BOTH tables
LEFT JOIN	All rows from left table + matches (NULL if no match)
RIGHT JOIN	All rows from right table + matches (NULL if no match)

```
-- INNER JOIN: only orders that have a valid seller
SELECT o.order_id, o.product, o.city AS customer_city, s.seller_name
FROM orders o
INNER JOIN sellers s ON o.seller_id = s.seller_id;

-- LEFT JOIN: all orders, even those with no seller (seller_name will be NULL)
SELECT o.order_id, o.product, s.seller_name
FROM orders o
LEFT JOIN sellers s ON o.seller_id = s.seller_id;

-- RIGHT JOIN: all sellers, even those with zero orders
SELECT s.seller_name, o.order_id, o.product
FROM orders o
RIGHT JOIN sellers s ON o.seller_id = s.seller_id;

-- JOIN + aggregation: total orders per seller
SELECT s.seller_name, COUNT(o.order_id) AS total_orders
FROM sellers s
LEFT JOIN orders o ON s.seller_id = o.seller_id
GROUP BY s.seller_name;
```

Output Explanation

LEFT JOIN keeps every row from the table on the LEFT of the keyword, filling unmatched columns from the right table with NULL. RIGHT JOIN does the opposite.

Common Interview Questions

Q: How is INNER JOIN different from LEFT JOIN?

A: INNER JOIN drops unmatched rows from both sides; LEFT JOIN keeps every row from the left table regardless of a match.

Q: How do you write a FULL OUTER JOIN in MySQL (it doesn't support FULL JOIN directly)?

A: Use `LEFT JOIN UNION RIGHT JOIN` to combine both unmatched sides.

Q: What is a self join?

A: A table joined to itself, used to compare rows within the same table (e.g. employees and their managers).

Beginner Mistake

Forgetting table aliases (o, s) in multi-table joins, causing "ambiguous column" errors when both tables share a column name like `city`.

Quick Revision - Chapter 8

- FOREIGN KEY enforces valid relationships between tables.
- ON DELETE CASCADE / SET NULL / RESTRICT control child-row behavior.
- INNER = matches only, LEFT = all left + matches, RIGHT = all right + matches.

Coding Practice - Chapter 8

- Write a LEFT JOIN to list every seller with their total revenue (0 if no orders).
- Write a query that finds sellers with zero orders (hint: LEFT JOIN + WHERE order_id IS NULL).

Chapter 9: Indexes & Views

9.1 Indexes

- An index = like a book's index page — helps MySQL find rows fast without scanning every row.
- Makes SELECT faster, but slows INSERT/UPDATE/DELETE (index must be updated too).
- Only index columns you frequently search/filter by.

```
CREATE INDEX idx_orders_city ON orders (city);           -- single-column index
CREATE INDEX idx_orders_city_status ON orders (city, order_status); -- composite index
SELECT * FROM orders WHERE city = 'Delhi';             -- now uses the index automatically
DROP INDEX idx_orders_city ON orders;                  -- removes index, data stays
```

When to Create Index	When NOT to Create Index
Columns in WHERE/JOIN frequently	Columns with very few unique values (e.g., gender)
Large tables with many reads	Tables that are frequently updated
Columns used in ORDER BY	Very small tables

9.2 Views

- A view = a saved SQL query that acts like a virtual table.
- Does NOT store data — it stores the query and runs it on demand.
- Use to simplify complex queries, hide sensitive columns, or reuse logic.

```
-- Create a view for delivered orders only
CREATE VIEW delivered_orders AS
SELECT order_id, customer_name, city, product, price_per_unit, order_date
FROM orders
WHERE order_status = 'Delivered';

-- Use the view exactly like a table
SELECT * FROM delivered_orders;

-- Update through a simple view (affects original table)
UPDATE delivered_orders SET price_per_unit = price_per_unit + 500 WHERE order_id = 1;

-- Drop the view (original table is untouched)
DROP VIEW delivered_orders;
```

Common Interview Questions

Q: What is the difference between a View and a Table?

A: A table stores actual data physically. A view stores only the query — data comes from the underlying tables at runtime.

Q: Can you always update data through a view?

A: Only if the view is simple (single table, no aggregation, no DISTINCT). Complex views are read-only.

Quick Revision — Chapter 9

- Index = speed up reads, slow down writes. Use wisely.
- View = virtual table = saved query. Great for simplification and security.

Chapter 10: GROUP BY, HAVING & ROLLUP

10.1 GROUP BY

- Groups rows with the same value in a column, then runs aggregate functions on each group.
- Every column in SELECT must be either in GROUP BY OR inside an aggregate function.

```
-- Orders count per city
SELECT city, COUNT(*) AS total_orders
FROM orders
GROUP BY city;

-- Total revenue per category
SELECT category, SUM(quantity * price_per_unit) AS total_sales
FROM orders
GROUP BY category;

-- Average price per city
SELECT city, AVG(price_per_unit) AS avg_price
FROM orders
GROUP BY city;

-- Group by two columns
SELECT city, order_status, COUNT(*) AS count
FROM orders
GROUP BY city, order_status
ORDER BY city, count DESC;
```

10.2 HAVING — Filtering Groups

WHERE vs HAVING

WHERE filters BEFORE grouping (individual rows).
HAVING filters AFTER grouping (whole groups).

```
-- Cities with more than 1 order
SELECT city, COUNT(*) AS total_orders
FROM orders
GROUP BY city
HAVING COUNT(*) > 1;

-- Categories with total sales above 50000
SELECT category, SUM(quantity * price_per_unit) AS total_sales
FROM orders
GROUP BY category
HAVING SUM(quantity * price_per_unit) > 50000
ORDER BY total_sales DESC;
```

10.3 ROLLUP — Auto Subtotals & Grand Total

```
SELECT city, category, SUM(quantity * price_per_unit) AS total_sales
FROM orders
GROUP BY city, category WITH ROLLUP;
-- (city, category) = normal grouped row
-- (city, NULL)      = subtotal for that city
-- (NULL, NULL)     = grand total of all sales
```

Memory Trick

NULL in a ROLLUP row = "this is a total row, not missing data."

Common Interview Questions

Q: Can you use WHERE with aggregate functions?

A: No. Use HAVING instead. WHERE cannot reference aggregate results like COUNT(*) or SUM().

Q: Execution order of SQL clauses?

A: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Beginner Mistake

Putting a non-aggregated, non-grouped column in SELECT — e.g., `SELECT city, product, COUNT(*) FROM orders GROUP BY city` — MySQL may return random values for `product`.

Quick Revision — Chapter 10

- GROUP BY + aggregate = summary per group.
- HAVING filters groups; WHERE filters rows.
- ROLLUP adds subtotals and a grand total row automatically.

Coding Practice — Chapter 10

1. Find the payment_mode with the highest total revenue.
2. List cities that have more than 2 delivered orders.
3. Show total sales per city with a grand total using ROLLUP.

Chapter 11: Subqueries & UNION

11.1 Subqueries

- A subquery = a SELECT inside another SELECT (runs first, result feeds outer query).
- Can be used in WHERE, SELECT, or FROM clauses.

```
-- Orders priced above average
SELECT * FROM orders
WHERE price_per_unit > (SELECT AVG(price_per_unit) FROM orders);

-- Orders from cities where Electronics were sold
SELECT * FROM orders
WHERE city IN (SELECT city FROM orders WHERE category = 'Electronics');

-- Show avg price alongside each order (subquery in SELECT)
SELECT order_id, customer_name, price_per_unit,
       (SELECT AVG(price_per_unit) FROM orders) AS avg_price
FROM orders;

-- EXISTS: orders from cities that also have Furniture orders
SELECT * FROM orders o
WHERE EXISTS (
    SELECT 1 FROM orders
    WHERE city = o.city AND category = 'Furniture'
);
```

Memory Trick

Think of a subquery like a calculator in brackets — MySQL solves the inner part first, then plugs the result into the outer query.

11.2 UNION & UNION ALL

- Stacks result sets from multiple SELECT queries vertically.
- All SELECT statements must have the same number and type of columns.

```
-- Create employee_orders table for demo
CREATE TABLE employee_orders (
    order_id      INT,
    name           VARCHAR(100),
    city           VARCHAR(50),
    product        VARCHAR(100),
    price_per_unit DECIMAL(10,2)
);

-- UNION: removes duplicate rows
SELECT order_id, customer_name AS name, city, product, price_per_unit FROM orders
UNION
SELECT order_id, name, city, product, price_per_unit FROM employee_orders;

-- UNION ALL: keeps all rows including duplicates (faster)
SELECT order_id, customer_name AS name, city, product, price_per_unit FROM orders
UNION ALL
SELECT order_id, name, city, product, price_per_unit FROM employee_orders;
```

	UNION	UNION ALL
Duplicates	Removed	Kept
Speed	Slower (dedup step)	Faster
Use when	You need unique results	You know no duplicates / want all rows

Common Interview Questions

Q: Difference between subquery and JOIN?

A: JOINS combine columns side-by-side; subqueries use one query's result as input to another. JOINS are usually faster; subqueries are more readable for certain logic.

Q: When to use EXISTS vs IN?

A: EXISTS stops as soon as one match is found (faster for large datasets); IN collects all matching values first. Prefer EXISTS for correlated subqueries.

Quick Revision — Chapter 11

- Subquery = query inside query. Inner runs first.
- UNION stacks result sets; UNION removes dupes, UNION ALL keeps them.

Chapter 12: Stored Procedures & Triggers

12.1 Stored Procedures

- A saved block of SQL that runs as one unit — reusable, reduces repetition.
- Must change the DELIMITER because the procedure contains multiple `;` statements.

```
DELIMITER //    -- change end-marker so MySQL doesn't split procedure early

CREATE PROCEDURE get_delivered_orders()
BEGIN
    SELECT * FROM orders WHERE order_status = 'Delivered';
END //

DELIMITER ;    -- always reset delimiter back

CALL get_delivered_orders();    -- runs the procedure
```

Procedure with Parameter

```
DELIMITER //

CREATE PROCEDURE get_orders_by_city(IN city_name VARCHAR(50))
BEGIN
    SELECT * FROM orders WHERE city = city_name;
END //

DELIMITER ;

CALL get_orders_by_city('Delhi');    -- pass city name as argument
```

Why Change DELIMITER?

MySQL sees `;` and thinks the statement is over. By switching to `//`, it knows to wait until it sees `//` before treating the whole procedure as one statement.

12.2 Triggers

- Triggers run automatically when a table event occurs (INSERT, UPDATE, DELETE).
- Cannot be called manually.
- Used for logging, audit trails, enforcing rules.

```
-- Step 1: create log table
CREATE TABLE order_cancellations (
    log_id        INT PRIMARY KEY AUTO_INCREMENT,
    order_id      INT,
    cancelled_on  DATETIME,
    reason        VARCHAR(100)
);

DELIMITER //

-- Step 2: create the trigger
CREATE TRIGGER trg_log_order_cancel
AFTER UPDATE ON orders    -- fires after any UPDATE on orders
FOR EACH ROW              -- runs for each row that was updated
BEGIN
    IF NEW.order_status = 'Cancelled'
    AND OLD.order_status <> 'Cancelled' THEN    -- only if status CHANGED to Cancelled
        INSERT INTO order_cancellations (order_id, cancelled_on, reason)
        VALUES (NEW.order_id, NOW(), 'Order cancelled by user');
    END IF;
```

```

END //

DELIMITER ;

-- Test: cancel order 2, then check log table
UPDATE orders SET order_status = 'Cancelled' WHERE order_id = 2;
SELECT * FROM order_cancellations;

```

Trigger Timing	Event	Keywords
BEFORE	Before the change is applied	NEW, OLD
AFTER	After the change is applied	NEW, OLD
Events: INSERT, UPDATE, DELETE. NEW = new values; OLD = previous values.		

Common Interview Questions

Q: What is the difference between a Stored Procedure and a Function in MySQL?

A: A stored procedure may return 0 or many results and is called with CALL. A function always returns exactly one value and can be used inside a SELECT.

Q: Can a trigger call another trigger?

A: Not directly in MySQL. Triggers cannot explicitly invoke other triggers, but a trigger action (like INSERT) on another table may fire that table's own trigger.

Quick Revision — Chapter 12

- Stored Procedure = reusable named SQL block, called with CALL.
- Always change DELIMITER before creating a procedure/trigger.
- Trigger = auto-runs on INSERT/UPDATE/DELETE. Use NEW/OLD for values.

Chapter 13: 100+ SQL Interview Questions & Answers

Section A: Conceptual Questions

1. What is SQL?

Structured Query Language — the standard language to communicate with relational databases to store, retrieve, update and delete data.

2. What are DDL, DML, DQL, DCL, TCL?

DDL (CREATE, ALTER, DROP), DML (INSERT, UPDATE, DELETE), DQL (SELECT), DCL (GRANT, REVOKE), TCL (COMMIT, ROLLBACK, SAVEPOINT).

3. What is a primary key?

A column (or set of columns) that uniquely identifies each row. Cannot be NULL or duplicate. One per table.

4. What is a foreign key?

A column that references the primary key of another table. Enforces referential integrity.

5. Difference between UNIQUE and PRIMARY KEY?

A table can have many UNIQUE constraints (each allows one NULL). It can have only ONE primary key (no NULLs at all).

6. What is a NULL value?

NULL = unknown/missing value. Not zero, not empty string. Always use IS NULL / IS NOT NULL to check for it.

7. What is normalization?

Process of organizing a database to reduce redundancy and improve data integrity by splitting data into related tables.

8. What are 1NF, 2NF, 3NF?

1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependency. 3NF: 2NF + no transitive dependency.

9. What is denormalization?

Intentionally adding redundancy to improve read performance (used in data warehouses).

10. What is ACID?

Atomicity (all-or-nothing), Consistency (valid state), Isolation (concurrent transactions don't interfere), Durability (committed data persists).

11. What is a join? Name all types.

JOIN combines rows from two tables. Types: INNER, LEFT, RIGHT, FULL OUTER (emulated in MySQL), CROSS, SELF.

12. What is a self join?

Joining a table to itself using aliases. Useful for hierarchical data (e.g., employee-manager relationship).

13. What is a cross join?

Returns the Cartesian product — every row from table A paired with every row from table B. No ON condition needed.

14. What is a subquery?

A SELECT query nested inside another query. Inner query runs first and its result is used by the outer query.

15. Correlated vs non-correlated subquery?

Non-correlated runs independently. Correlated refers to the outer query's columns and re-runs for each outer row.

16. What is a view?

A virtual table built from a stored SQL query. Does not store data itself. Updated when queried.

17. Can we update data through a view?

Only if the view is based on a single table with no aggregation, DISTINCT, or GROUP BY.

18. What is an index?

A data structure that speeds up SELECT queries at the cost of slower writes. Like an index page in a book.

19. What is a clustered vs non-clustered index?

Clustered: data rows sorted/stored in index order (one per table, usually the primary key). Non-clustered: separate structure pointing to data rows.

20. What is a stored procedure?

A pre-compiled named block of SQL stored in the database and executed with CALL. Reduces repetition and network traffic.

21. What is a trigger?

SQL code that runs automatically on INSERT/UPDATE/DELETE events on a table. Cannot be called manually.

22. What is a transaction?

A group of SQL statements that execute as one unit — all succeed or all fail together.

23. What is the difference between DELETE and TRUNCATE?

DELETE removes rows one by one, supports WHERE, can be rolled back. TRUNCATE removes all rows at once, faster, resets AUTO_INCREMENT, cannot be rolled back.

24. What is the difference between TRUNCATE and DROP?

TRUNCATE empties the table (structure stays). DROP removes the table entirely (structure + data gone).

25. What is a composite key?

A primary key made of two or more columns combined — used when no single column is unique by itself.

26. What is a candidate key?

Any column or set of columns that could serve as a primary key (uniquely identifies rows, no NULLs).

27. What is a surrogate key?

An artificial primary key (like AUTO_INCREMENT id) with no business meaning, created purely for identification.

28. What is referential integrity?

Ensures a foreign key value always exists in the parent table — no orphan records allowed.

29. What does ON DELETE CASCADE do?

When a parent row is deleted, all child rows referencing it are automatically deleted too.

30. What is the difference between CHAR and VARCHAR?

CHAR is fixed-length (always uses the declared size, pads with spaces). VARCHAR is variable-length (uses only what's needed). VARCHAR saves space for varying text.

Section B: Query / Coding Questions

31. Find the 2nd highest price_per_unit from orders.

```
SELECT MAX(price_per_unit) FROM orders
WHERE price_per_unit < (SELECT MAX(price_per_unit) FROM orders);
-- OR using LIMIT/OFFSET
SELECT DISTINCT price_per_unit FROM orders ORDER BY price_per_unit DESC LIMIT 1 OFFSET 1;
```

32. Find duplicate customer_names in orders.

```
SELECT customer_name, COUNT(*) AS cnt
FROM orders
GROUP BY customer_name
HAVING COUNT(*) > 1;
```

33. Delete duplicate rows keeping only the one with the lowest order_id.

```
DELETE FROM orders
WHERE order_id NOT IN (
    SELECT MIN(order_id) FROM orders GROUP BY customer_name
);
```

34. Count NULL values in delivery_date.

```
SELECT COUNT(*) - COUNT(delivery_date) AS null_count FROM orders;
-- COUNT(*) counts all rows; COUNT(col) skips NULLs
```

35. Find total revenue per category sorted high to low.

```
SELECT category, SUM(quantity * price_per_unit) AS revenue
FROM orders
GROUP BY category
ORDER BY revenue DESC;
```

36. Find orders where delivery took more than 5 days.

```
SELECT order_id, customer_name, DATEDIFF(delivery_date, order_date) AS days
```

```
FROM orders
WHERE DATEDIFF(delivery_date, order_date) > 5;
```

37. Find the city with the most orders.

```
SELECT city, COUNT(*) AS total
FROM orders
GROUP BY city
ORDER BY total DESC LIMIT 1;
```

38. Show orders placed in January 2025.

```
SELECT * FROM orders
WHERE YEAR(order_date) = 2025 AND MONTH(order_date) = 1;
```

39. List customers who never gave a rating (rating IS NULL).

```
SELECT customer_name, product FROM orders WHERE rating IS NULL;
```

40. Calculate discounted price for each order.

```
SELECT order_id, product, price_per_unit,
       price_per_unit * (1 - discount_percent / 100.0) AS discounted_price
FROM orders;
```

41. Find sellers with no orders (using LEFT JOIN).

```
SELECT s.seller_name
FROM sellers s
LEFT JOIN orders o ON s.seller_id = o.seller_id
WHERE o.order_id IS NULL;
```

42. Rank orders by price within each category (using window function).

```
SELECT category, product, price_per_unit,
       RANK() OVER (PARTITION BY category ORDER BY price_per_unit DESC) AS rnk
FROM orders;
```

43. Show running total of price_per_unit ordered by order_date.

```
SELECT order_id, order_date, price_per_unit,
       SUM(price_per_unit) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

44. Find orders where price is above the category average.

```
SELECT o.* FROM orders o
WHERE price_per_unit > (
    SELECT AVG(price_per_unit) FROM orders WHERE category = o.category
);
```

45. Swap values of two columns in a single UPDATE.

```
UPDATE orders SET order_date = delivery_date, delivery_date = order_date
WHERE order_id = 1; -- MySQL handles this correctly in a single statement
```

46. Find the percentage of delivered orders.

```
SELECT
    SUM(CASE WHEN order_status = 'Delivered' THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS pct_delivered
FROM orders;
```

47. List products that appear in more than one order.

```
SELECT product, COUNT(*) AS times_ordered
FROM orders GROUP BY product HAVING COUNT(*) > 1;
```

48. Retrieve top 3 categories by total revenue.

```
SELECT category, SUM(quantity * price_per_unit) AS revenue
FROM orders GROUP BY category ORDER BY revenue DESC LIMIT 3;
```

49. Find orders placed on weekends.

```
SELECT * FROM orders
WHERE DAYOFWEEK(order_date) IN (1, 7); -- 1=Sunday, 7=Saturday
```

50. Convert order_status to a number for sorting (Delivered=1, Pending=2, Cancelled=3).

```
SELECT *, CASE order_status
  WHEN 'Delivered' THEN 1
  WHEN 'Pending'   THEN 2
  ELSE 3
END AS status_order
FROM orders ORDER BY status_order;
```

Section C: More Interview Q&A (51-100)

51. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

ROW_NUMBER: unique sequential number, no ties. RANK: same rank for ties, skips next number. DENSE_RANK: same rank for ties, no gaps.

52. What is a CTE (Common Table Expression)?

A temporary named result set using WITH ... AS () that can be referenced in the same query. Makes complex queries more readable.

53. What is a window function?

A function that performs calculations across a "window" of rows related to the current row, without collapsing rows like GROUP BY does. Uses OVER() clause.

54. What is COALESCE()?

Returns the first non-NULL value from a list. COALESCE(rating, 0) returns rating if not NULL, else 0.

55. What is IFNULL() in MySQL?

IFNULL(col, default) — returns default if col is NULL. Similar to COALESCE but only takes 2 arguments.

56. What is the ORDER of SQL execution?

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

57. Can we use column alias in WHERE clause?

No. WHERE is processed before SELECT, so aliases don't exist yet. Use HAVING or wrap in a subquery.

58. What is the difference between IN and EXISTS?

IN evaluates all subquery results first. EXISTS stops as soon as one match is found — more efficient for large correlated subqueries.

59. How do you find the Nth highest value?

Use DENSE_RANK() window function or LIMIT + OFFSET: `SELECT DISTINCT col FROM t ORDER BY col DESC LIMIT 1 OFFSET N-1`

60. What is a materialized view?

A view that stores results physically (unlike a regular view). MySQL doesn't natively support them; PostgreSQL does.

61. What is a deadlock?

When two transactions wait for each other's lock to release, causing both to freeze. MySQL detects and rolls back one of them.

62. What is the CASE statement?

SQL's if-else logic: `CASE WHEN condition THEN result ... ELSE default END`

63. How to pivot data in MySQL?

Use CASE + SUM/COUNT with GROUP BY to simulate pivot (MySQL has no native PIVOT keyword).

64. What is GROUP_CONCAT() in MySQL?

Concatenates values from multiple rows into a single string: `GROUP_CONCAT(product SEPARATOR ',')`

65. What is the difference between SAVEPOINT and ROLLBACK?

SAVEPOINT marks a point you can roll back to partially. ROLLBACK without a SAVEPOINT undoes the whole transaction.

66. What is an auto-increment reset?

`ALTER TABLE orders AUTO_INCREMENT = 1;` — resets counter (only works if current max id allows it).

67. What is EXPLAIN in MySQL?

Shows how MySQL will execute a query — which indexes it uses, estimated rows scanned. Used for query optimization.

68. What is a covering index?

An index that contains all columns the query needs — MySQL can answer the query from the index alone without touching the table data.

69. How many types of relationships exist in RDBMS?

One-to-One, One-to-Many (most common), Many-to-Many (implemented via a junction/bridge table).

70. What is a junction table?

A table that resolves a many-to-many relationship by holding foreign keys from both parent tables (e.g., orders_products).

71. What is UNION used for?

To combine result sets of two or more SELECT queries vertically. Column count and types must match.

72. What is the difference between UNION and JOIN?

UNION stacks rows vertically. JOIN combines columns horizontally from two tables.

73. What is INTERSECT? Does MySQL support it?

Returns rows common to both queries. MySQL 8.0+ supports it. Simulate in older MySQL: use INNER JOIN or IN with subquery.

74. What is EXCEPT / MINUS?

Returns rows in first query that don't appear in second. MySQL calls it EXCEPT (supported from 8.0.31). Use LEFT JOIN + IS NULL to simulate.

75. How do you copy a table structure without data?

`CREATE TABLE new_table LIKE old_table;`

76. How do you copy a table with data?

`CREATE TABLE new_table AS SELECT * FROM old_table;`

77. How to add a column with a default value to an existing table?

`ALTER TABLE orders ADD COLUMN is_gift BOOLEAN DEFAULT FALSE;`

78. Can a table have no primary key?

Yes, but it's bad practice. Without a PK, duplicate rows can exist and performance suffers.

79. What is a natural join?

Joins two tables automatically on columns with the same name and compatible types. Avoid in production — fragile and implicit.

80. What is the HAVING clause vs WHERE?

WHERE filters before aggregation (individual rows). HAVING filters after aggregation (groups).

81. What is BETWEEN and is it inclusive?

BETWEEN a AND b checks a <= value <= b — yes, inclusive on both ends.

82. How to find records in table A not in table B?

`SELECT * FROM A LEFT JOIN B ON A.id = B.id WHERE B.id IS NULL`

83. What does DISTINCT do?

Removes duplicate rows from the SELECT result. Applied after SELECT.

84. Can DISTINCT work on multiple columns?

Yes — `SELECT DISTINCT city, category FROM orders` returns unique city+category combinations.

85. How to get current date and time in MySQL?

`SELECT CURRENT_DATE, CURRENT_TIME, NOW();`

86. What is FLOOR(), CEIL(), ROUND()?

FLOOR rounds down, CEIL rounds up, ROUND rounds to nearest (with optional decimal places).

87. What is CONCAT() in MySQL?

Joins strings: `CONCAT(first_name, ' ', last_name)`

88. What is TRIM()?

Removes leading and trailing spaces: `TRIM(' hello ') = 'hello'`

89. What is SUBSTRING()?

Extracts part of a string: `SUBSTRING('Delhi', 1, 3) = 'Del'`

90. What is REPLACE() in SQL?

Replaces all occurrences of a substring: `REPLACE('Hello World', 'World', 'SQL')`

91. How do you check the MySQL version?

```
SELECT VERSION();
```

92. What is the difference between MyISAM and InnoDB?

InnoDB supports transactions, foreign keys, row-level locking. MyISAM is faster for reads but no transactions/FK support. InnoDB is MySQL's default and recommended engine.

93. How do you find the size of a MySQL database?

```
SELECT table_schema, SUM(data_length+index_length)/1024/1024 AS size_mb FROM information_schema.tables GROUP BY table_schema;
```

94. What is a recursive CTE?

A CTE that refers to itself — used for hierarchical/tree data like org charts or bill of materials.

95. What is the difference between INNER JOIN and WHERE with two tables?

Functionally same for equality joins, but explicit JOIN syntax is clearer, more readable, and required for OUTER joins.

96. What is LAG() and LEAD() window function?

LAG accesses the previous row's value; LEAD accesses the next row's value — both within a defined window/partition.

97. What is PARTITION BY in window functions?

Divides the dataset into groups for window calculations, similar to GROUP BY but without collapsing rows.

98. What is the default sort order of ORDER BY?

ASC (ascending) is the default. Specify DESC explicitly for descending.

99. How to update multiple tables in one query?

Use a multi-table UPDATE with JOIN: `UPDATE t1 JOIN t2 ON ... SET t1.col = val WHERE ...`

100. What makes a good SQL query (performance)?

Use indexes on filtered columns, avoid SELECT *, limit rows early with WHERE, avoid functions on indexed columns in WHERE, prefer JOINS over subqueries where possible, use EXPLAIN to profile.

Chapter 14: Cheat Sheets & Quick Reference

14.1 SQL Execution Order

Step	Clause	What it does
1	FROM / JOIN	Identifies tables and joins them
2	WHERE	Filters individual rows
3	GROUP BY	Groups remaining rows
4	HAVING	Filters groups
5	SELECT	Picks columns, computes expressions
6	DISTINCT	Removes duplicate rows
7	ORDER BY	Sorts the result
8	LIMIT / OFFSET	Cuts down number of rows returned

Memory Trick: "From Where Groups Have Selected Distinct Order Limits"

F-W-G-H-S-D-O-L

14.2 Aggregate Functions Cheat Sheet

Function	Returns	NULL handling
COUNT(*)	All rows including NULLs	Counts NULLs
COUNT(col)	Non-NULL rows in that column	Skips NULLs
SUM(col)	Total of all values	Ignores NULLs
AVG(col)	Average of non-NULL values	Ignores NULLs
MIN(col)	Smallest value	Ignores NULLs
MAX(col)	Largest value	Ignores NULLs
GROUP_CONCAT(col)	Concatenates values into string	Ignores NULLs

14.3 JOIN Cheat Sheet

JOIN Type	Returns	Syntax
INNER JOIN	Matched rows only	FROM A INNER JOIN B ON A.id = B.id
LEFT JOIN	All A + matched B (NULL if no match)	FROM A LEFT JOIN B ON ...
RIGHT JOIN	All B + matched A (NULL if no match)	FROM A RIGHT JOIN B ON ...
FULL OUTER JOIN	All rows from both (MySQL: LEFT UNION RIGHT)	No native syntax in MySQL
CROSS JOIN	Every A row × every B row	FROM A CROSS JOIN B
SELF JOIN	Table joined to itself	FROM A a1 JOIN A a2 ON a1.mgr = a2.id

14.4 Window Functions Cheat Sheet

Function	Use	Example
ROW_NUMBER()	Unique row number (no ties)	ROW_NUMBER() OVER (ORDER BY price DESC)
RANK()	Rank with gaps on ties	RANK() OVER (ORDER BY price DESC)

DENSE_RANK()	Rank without gaps on ties	DENSE_RANK() OVER (ORDER BY price DESC)
SUM() OVER()	Running total	SUM(price) OVER (ORDER BY order_date)
AVG() OVER()	Moving average	AVG(price) OVER (PARTITION BY category)
LAG(col, n)	Value from n rows before	LAG(price, 1) OVER (ORDER BY order_date)
LEAD(col, n)	Value from n rows after	LEAD(price, 1) OVER (ORDER BY order_date)
FIRST_VALUE()	First value in window	FIRST_VALUE(price) OVER (PARTITION BY cat ORDER BY price)
NTILE(n)	Divides into n buckets	NTILE(4) OVER (ORDER BY price) — quartiles

14.5 Common Errors & Fixes

Error	Cause	Fix
No database selected	Forgot USE dbname	Run USE ecom;
Unknown column in WHERE	Using SELECT alias in WHERE	Repeat expression or use subquery/HAVING
Duplicate entry for key PRIMARY	Inserting existing PK value	Remove the id from INSERT or use AUTO_INCREMENT
Cannot add/update a child row: FK constraint fails	FK value doesn't exist in parent	Insert parent row first
Data too long for column	VARCHAR size too small	ALTER COLUMN to increase size
You are using safe update mode	UPDATE/DELETE without WHERE	Add WHERE clause or SET SQL_SAFE_UPDATES=0;
Every derived table must have its own alias	Subquery in FROM without alias	Add AS alias after the closing parenthesis
Incorrect datetime value	Wrong date format	Use 'YYYY-MM-DD' format

14.6 Most Asked SQL Queries in Interviews

```
-- 1. Nth highest value
SELECT DISTINCT price_per_unit FROM orders ORDER BY price_per_unit DESC LIMIT 1 OFFSET N-1;

-- 2. Duplicates
SELECT customer_name, COUNT(*) FROM orders GROUP BY customer_name HAVING COUNT(*) > 1;

-- 3. Records in A not in B
SELECT * FROM A LEFT JOIN B ON A.id = B.id WHERE B.id IS NULL;

-- 4. Running total
SELECT order_id, SUM(price_per_unit) OVER (ORDER BY order_date) FROM orders;

-- 5. Rank within group
SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY price_per_unit DESC) AS rnk
FROM orders;

-- 6. Pivot with CASE
SELECT
    city,
    SUM(CASE WHEN category = 'Electronics' THEN quantity * price_per_unit ELSE 0 END) AS electronics_rev,
    SUM(CASE WHEN category = 'Furniture' THEN quantity * price_per_unit ELSE 0 END) AS furniture_rev
FROM orders GROUP BY city;

-- 7. Delete duplicates keeping lowest id
DELETE FROM orders WHERE order_id NOT IN
    (SELECT MIN(order_id) FROM orders GROUP BY customer_name);

-- 8. Month-wise sales
SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mo,
    SUM(quantity * price_per_unit) AS revenue
```

```
FROM orders GROUP BY yr, mo ORDER BY yr, mo;
```

14.7 SQL Syntax Summary (1-Page Revision)

Command	Quick Syntax
Create DB	CREATE DATABASE name;
Create Table	CREATE TABLE t (col type constraints, ...);
Insert	INSERT INTO t (cols) VALUES (vals), (...);
Select	SELECT cols FROM t WHERE cond ORDER BY col LIMIT n;
Update	UPDATE t SET col=val WHERE cond;
Delete	DELETE FROM t WHERE cond;
Group	SELECT col, AGG FROM t GROUP BY col HAVING cond;
Join	SELECT ... FROM A JOIN B ON A.id = B.id;
Subquery	SELECT ... WHERE col IN (SELECT col FROM t WHERE ...);
Union	SELECT ... UNION [ALL] SELECT ...;
Window	FUNC() OVER (PARTITION BY col ORDER BY col);
View	CREATE VIEW vname AS SELECT ...;
Index	CREATE INDEX idx ON t(col);
Procedure	CREATE PROCEDURE p() BEGIN ... END;
Trigger	CREATE TRIGGER t AFTER UPDATE ON tbl FOR EACH ROW BEGIN ... END;
Transaction	SET autocommit=0; ...changes... COMMIT; or ROLLBACK;
Alter	ALTER TABLE t ADD/MODIFY/DROP/RENAME COLUMN col ...;
Drop	DROP TABLE [IF EXISTS] t;

Final Interview Tips

- Always think: "Which rows? Which columns? In what order?" before writing any query.
- For "top N" questions: ORDER BY DESC + LIMIT N.
- For "not in another table": LEFT JOIN + WHERE right_id IS NULL.
- For "within group ranking": window functions with PARTITION BY.
- NULL is your best friend AND worst enemy — always check with IS NULL.
- Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.